

ABU DHABI UNIVERSITY

College of Engineering

SWE401 - Software Engineering

Course Project - Phase 5

Implementation and Testing

Project Title

SmartStudy - Intelligent Academic Planning System

NO.	STUDENT NAME	STUDENT ID
1.	Ismail Monem	1096000
2.	Ahmed El Jeyid	1093121
3.	Mohamed Ahmed	1091429

Instructor

Dr. Chayanika Sharma

Semester

Summer 2025-2026

Submission Date

3 August 2026

Table of Contents

8. Implementation and Testing	3
8.1 Implementation Overview	3
8.2 Implemented System Architecture	4
8.3 Functional Implementation	5
8.4 Database and Smart Features	7
8.5 Source Control, Build and Run Instructions	8
8.6 Testing and Verification	9
8.7 Implementation Outputs	10
8.8 Known Limitations and Final Validation	17
8.9 Team Contributions	18
References	18

8. Implementation and Testing

8.1 Implementation Overview

SmartStudy was implemented as a Java desktop application using JavaFX for the graphical user interface, Java for application logic, MySQL for persistent storage, and JDBC for database access. The implementation translates the requirements and design models from Phases 2-4 into a structured Maven project and was executed successfully on the team workstation.

This Phase 5 report presents the completed implementation, source-control information, build and run instructions, automated test results, end-to-end verification, and actual application outputs. The application package contains source code, FXML layouts, CSS, SQL scripts, automated tests, sample syllabi, configuration templates, and a README file.

Category	Implementation
Programming language	Java 21 target; tested on Java 26
User interface	JavaFX with FXML and CSS
Database	MySQL 8+ accessed through JDBC
Build system	Apache Maven
Security	BCrypt password hashing; active-account checks; session invalidation
Document processing	Apache PDFBox for PDF and Apache POI for DOCX
Testing	JUnit 5 automated tests plus manual end-to-end verification
Version control	Git and GitHub; private repository published

Implementation Objectives

- Preserve the layered architecture approved in Phase 4.
- Implement the complete student flow from registration to progress tracking.
- Keep administrator account-management functions separate from academic data.
- Support representative PDF/DOCX syllabus extraction with mandatory student review.
- Generate and regenerate a rolling seven-day study plan using grade weight, deadline proximity, estimated effort, and available hours.
- Store all account, course, task, schedule, progress, and notification data in MySQL.

8.2 Implemented System Architecture

The source code follows the five-layer design from Phase 4. JavaFX views communicate with controllers, controllers call service classes, services apply business rules, DAOs perform prepared JDBC operations, and MySQL/local file storage persist the results. External LMS and email capabilities remain replaceable adapter boundaries; the prototype uses manual PDF/DOCX upload and in-app reminders.

Layer	Packages / Files	Responsibility
Presentation	resources/fxml, resources/css	Login, registration, dashboard, courses/tasks, syllabus review, schedule, profile, and administrator screens.
Control	controller	Handles UI events, validation feedback, screen navigation, and user commands.
Application logic	service	Authentication, session control, parsing, scheduling, reminder creation, progress and integration coordination.
Data access	dao	Prepared JDBC queries and transactions for every database table.
Persistence & integration	MySQL, data/syllabi	Relational records, referential integrity, uploaded files, and simulated external-service boundaries.

Project Structure

- src/main/java/com/smartstudy/model - domain entities and enumerations
- src/main/java/com/smartstudy/controller - JavaFX controllers
- src/main/java/com/smartstudy/service - business logic and smart features
- src/main/java/com/smartstudy/dao - JDBC database access
- src/main/resources/fxml - interface layouts
- src/main/resources/css - shared visual style
- database - schema and seed scripts
- sample-syllabi - representative PDF and DOCX files
- src/test - JUnit tests
- tools - source verification and core smoke test

The complete source package is delivered through the private GitHub repository and the final SmartStudy_Phase5_Final_Submission.zip file.

8.3 Functional Implementation

FR	Implemented behavior	Primary implementation
FR1	Registers a student after required-field, email, duplicate-email, password, and available-hours validation.	RegisterController, AuthService, StudentDao
FR2	Authenticates students and administrators; blocks incorrect credentials and deactivated accounts.	LoginController, AuthService
FR3	Updates full name, password, weekly available hours, and notification preferences.	ProfileController, StudentDao
FR4	Adds, edits, lists, and deletes courses belonging to the signed-in student.	CourseTaskController, CourseDao
FR5	Adds, edits, deletes, filters and completes assignments, exams and quizzes.	CourseTaskController, TaskDao
FR6	Copies PDF/DOCX syllabi into controlled local storage and records the file in MySQL.	SyllabusController, SmartStudyService
FR7	Extracts assessment titles, types, dates and weights; lets the student edit and confirm each item.	SyllabusParser, SyllabusDao
FR8	Creates future study sessions across a rolling seven-day window using task priority and available time.	ScheduleGenerator, ScheduleDao
FR9	Regenerates the remaining plan after task creation, edit, deletion, status change, or available-hours update.	SmartStudyService
FR10	Stores Pending, In Progress and Completed status; completed tasks are excluded from future plans and progress updates immediately.	TaskDao, DashboardController
FR11	Displays course count, weekly study time, progress, upcoming tasks and reminders.	DashboardController
FR12	Creates in-app deadline and study-session reminders when a plan is generated.	NotificationDao, SmartStudyService
FR13	Persists data through MySQL tables with foreign keys, unique constraints and cascading deletion.	database/schema.sql and DAOs
FR14	Clears the active session on logout and checks the inactivity timeout on protected operations.	SessionManager, MainController
FR15	Allows administrators to view, activate, deactivate and delete student accounts.	AdminController, StudentDao

Main User Flow

- The student creates an account and logs in.
- The student adds courses and tasks or uploads a representative syllabus.
- Extracted items are reviewed and corrected before confirmation.
- The system ranks active tasks and generates future study blocks across a rolling seven-day planning window.
- The dashboard displays deadlines, study hours, progress and reminders.
- Changing task information automatically regenerates the plan.
- The student logs out; protected data is inaccessible until a new login.

Administrator Flow

- The administrator signs in using the separate administrator account.
- The application displays only the Administrator Console and Logout actions.
- The administrator can activate, deactivate or permanently delete a selected student account.
- Academic courses, tasks and schedules are not exposed to the administrator interface.

Interface Consistency

All screens share the same blue application header, left navigation panel, typography, table styling and validation feedback. Student and administrator navigation are role-specific so a student cannot open the administrator console.

8.4 Database and Smart Features

Database Implementation

The database script creates nine tables: Student, Administrator, Course, Task, Syllabus, ExtractedItem, Schedule, StudySession, and Notification. The SQL implementation uses the corrected Phase 4 design, including courseId as a surrogate primary key, UNIQUE(studentId, courseCode), Student.isActive, Syllabus.storedPath, and ExtractedItem.title/dueDate.

Constraint	Purpose
UNIQUE students.email	Prevents duplicate accounts.
UNIQUE administrators.email	Prevents duplicate administrator identities.
UNIQUE(student_id, course_code)	Stops one student from adding the same course twice.
UNIQUE syllabi.course_id	Allows at most one stored syllabus per course.
Foreign keys with cascading deletion	Prevents orphaned academic records after course or account deletion.
Task status ENUM	Restricts status to PENDING, IN_PROGRESS or COMPLETED.
Indexes on deadlines/status/reminders	Supports responsive schedule and dashboard queries.

Schedule Generation

For each active task, the scheduler calculates a priority score from grade weight, urgency, and estimated effort. Tasks are sorted from highest to lowest score and divided into practical study blocks of up to 1.5 hours. The rolling seven-day plan begins on the current day, excludes past times and completed tasks, respects the student's weekly available-hours limit, and never schedules a block after its task deadline.

$$\text{priority} = 0.50(\text{weight}) + 0.35(\text{urgency}) + 0.15(\text{effort})$$

If the available time or the remaining time before a deadline is insufficient, the generator returns a clear warning rather than silently producing an impossible schedule.

Syllabus Extraction

The parser reads PDF text with PDFBox and DOCX paragraphs with Apache POI. It recognizes representative assessment keywords, common date formats and percentage weights. Because syllabus formats vary, no extracted item is converted into a task until the student reviews and confirms it.

Reminder Creation

Each regeneration replaces future automatic reminders and creates an in-app reminder before every generated study session and active task deadline. The database stores message, time, channel and sent status so a real notification gateway can be connected later.

8.5 Source Control, Build and Run Instructions

Repository

Repository URL: https://github.com/MohamedAhmedAdu/SmartStudy_Phase5_Final_Improved/tree/main/SmartStudy_Phase5_Final_Improved

The project was published as a private GitHub repository. Ahmed El Jeyid and Ismail Monem were invited as collaborators. Before final submission, each member must accept the invitation and make at least one genuine, visible contribution that matches the team-contribution statement.

Repository item	Purpose / status
main	Stable demonstration-ready version published to GitHub.
Ahmed contribution	Genuine UI review/documentation or related work committed from Ahmed's account.
Ismail contribution	Genuine syllabus/scheduler testing documentation or related work committed from Ismail's account.
Pull requests / review	Changes are reviewed before merging into main.
Private access	Collaborators and instructor must be invited before submission.

Prerequisites

- JDK 21 or later (tested using Java 26)
- Apache Maven 3.9 or later
- MySQL 8 or later
- Internet access during the first Maven build so required libraries can be downloaded

Database Setup

- Run database/schema.sql in MySQL.
- Run database/seed.sql.
- Copy smartstudy.properties.example in the project root.
- Rename the copy to smartstudy.properties and enter the local MySQL password. This private file is excluded from GitHub by .gitignore.

Build and Run

mvn clean test

mvn javafx:run

NetBeans users can open the folder as a Maven project, allow dependencies to download, and run com.smartstudy.SmartStudyApp.

Default administrator for demonstration: admin@smartstudy.local / Admin123!

8.6 Testing and Verification

The package includes JUnit tests, a dependency-free core smoke test, representative PDF/DOCX files, and a source-verification script. The following checks were executed while preparing the package:

Check	Result	Evidence
Automated JUnit suite	PASS	7 tests executed; 0 failures; 0 errors; 0 skipped; BUILD SUCCESS.
Schedule generation	PASS	Rolling seven-day schedules generated for multiple students and task sets.
Completed-task exclusion	PASS	Completed tasks were removed from upcoming tasks and future study sessions.
Syllabus extraction	PASS	PDF/DOCX assessments were extracted and presented for edit, discard, or confirmation.
Full JavaFX + MySQL execution	PASS	Application connected to MySQL and all major screens executed successfully.
Multi-account persistence	PASS	Three student accounts retained separate courses, tasks, progress, and schedules.
Administrator operations	PASS	Administrator viewed, activated, deactivated, and deleted student accounts.

Required End-to-End Test Cases

ID	Test	Verified result
TC-01	Register valid student	Account stored and login available.
TC-02	Register duplicate email	Clear rejection; no duplicate row.
TC-03	Student login/logout	Dashboard opens; logout blocks protected screens.
TC-04	Deactivated login	Access rejected.
TC-05	Course CRUD	List updates and persists after restart.
TC-06	Task CRUD/status	Task persists; dashboard and schedule update.
TC-07	Upload valid PDF/DOCX	File stored and items displayed for review.
TC-08	Upload unsupported/corrupt file	Error shown; no invalid record committed.
TC-09	Edit/confirm extracted item	Confirmed item becomes a Task.
TC-10	Generate schedule	Study sessions created in no more than 5 seconds.
TC-11	Automatic regeneration	Task change replaces current plan.
TC-12	Progress update	Completed task immediately changes progress.
TC-13	Reminder generation	Deadline and session reminders are persisted.
TC-14	Persistence/restart	Saved academic data remains available.
TC-15	Administrator operations	View, activate, deactivate and delete work correctly.

TC-01 through TC-15 were verified on the configured team workstation. The automated suite completed with seven tests, zero failures, zero errors, and BUILD SUCCESS. Actual execution screenshots are included in Section 8.7. Email delivery and institutional LMS integration were not claimed as tested because those external services were not configured.

8.7 Implementation Outputs

The following figures are actual outputs captured from the completed SmartStudy application. They demonstrate authentication, student-specific dashboards, course and task management, syllabus extraction, schedule generation, profile settings, administrator account management, and the successful automated test run.

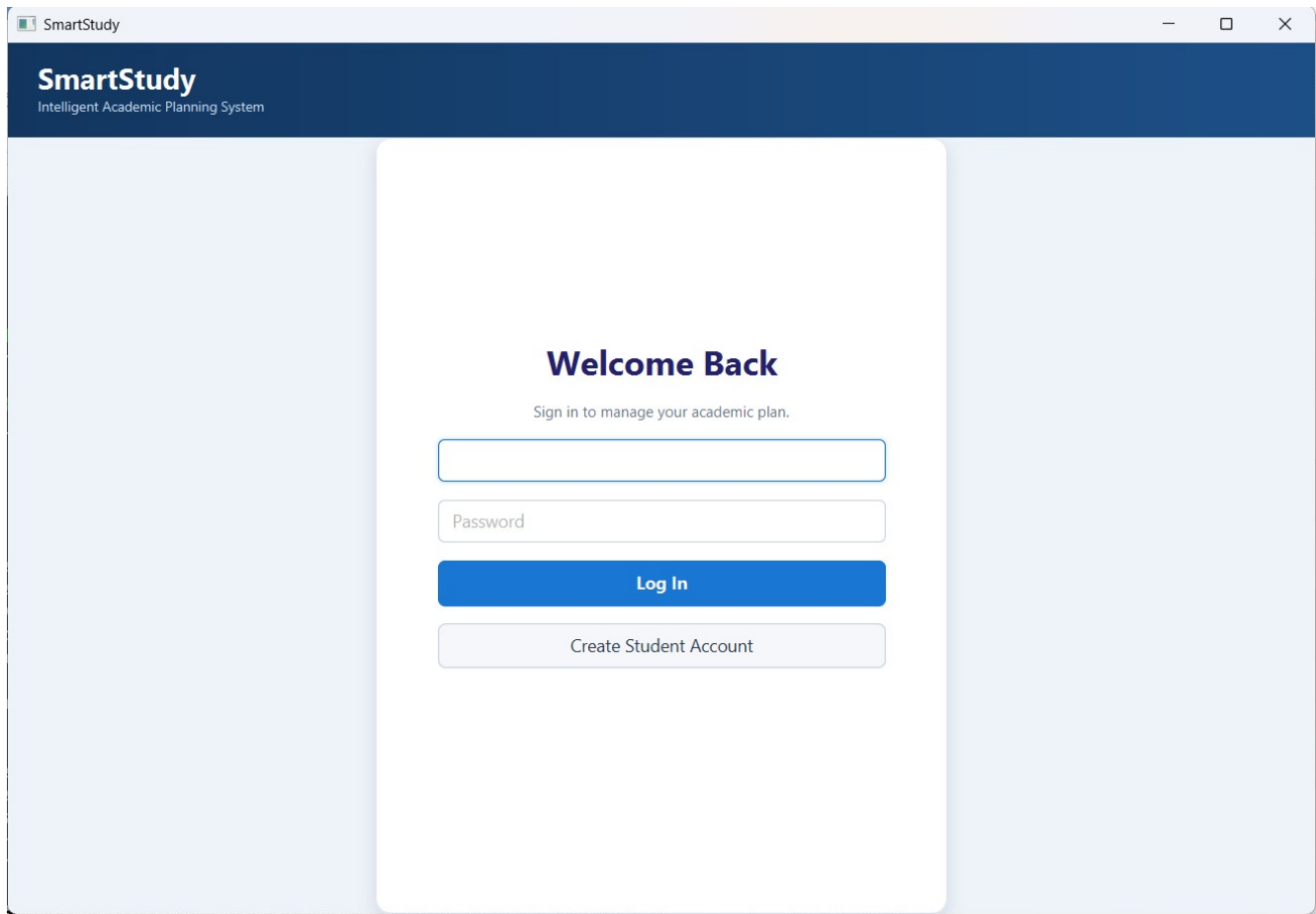


Figure 1. SmartStudy login interface for student and administrator authentication.

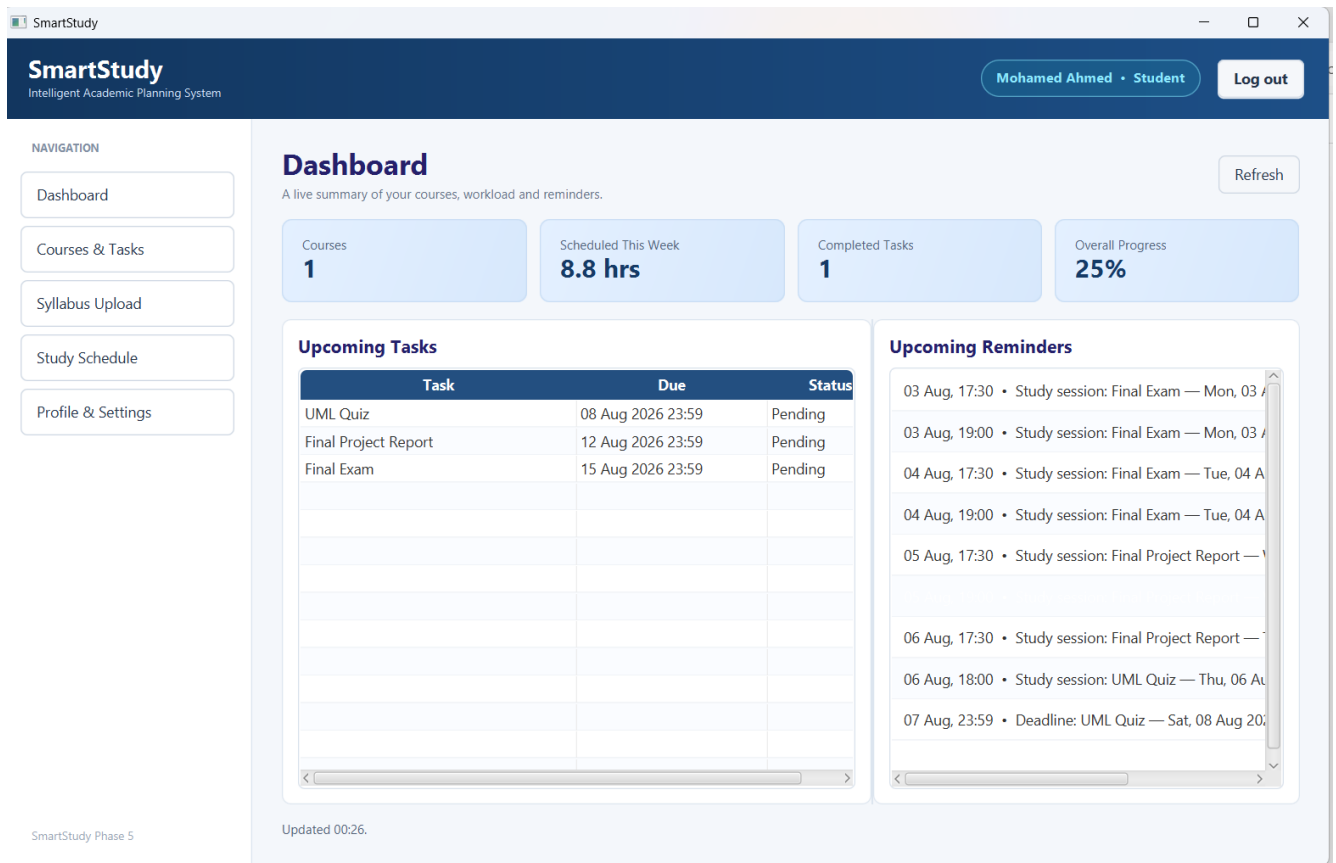


Figure 2. Mohamed Ahmed dashboard showing course count, scheduled hours, completed tasks, progress, upcoming tasks, and reminders.

SmartStudy

Intelligent Academic Planning System

Mohamed Ahmed • Student

Log out

NAVIGATION

Dashboard

Courses & Tasks

Syllabus Upload

Study Schedule

Profile & Settings

Courses & Tasks

Manage academic data manually or review tasks created from a syllabus.

Add Course

Edit Course

Delete Course

Code	Course Name	Instructor	Semester	
SWE401	Software Engineering	Dr.Chayanika Sharma	Summer 25 -26	

Add Task

Edit Task

Mark In Progress

Mark Completed

Delete Task

Task	Type	Due Date	Weight	Effort	Status	
Assignment	Assignment	06 Aug 2026	15.0%	1.5 h	Completed	
UML Quiz	Quiz	08 Aug 2026	10.0%	0.7 h	Pending	
Final Project Report	Assignment	12 Aug 2026	30.0%	3.0 h	Pending	
Final Exam	Exam	15 Aug 2026	40.0%	5.0 h	Pending	

SmartStudy Phase 5

1 course(s) loaded.

Figure 3. Courses & Tasks screen showing CRUD controls, grade weights, estimated effort, and task status.

SmartStudy

Intelligent Academic Planning System

Ahmed El Jeyid • Student

Log out

NAVIGATION

Dashboard

Courses & Tasks

Syllabus Upload

Study Schedule

Profile & Settings

Syllabus Upload & Extraction

SmartStudy detects likely assessments. You remain in control: edit, discard or confirm every item.

Course:

CSC202 - Database Mangement

Choose PDF or DOCX

No file selected

Extracted Item	Type	Due Date	Weight	
Final Exam Will examine students' understanding of all course	Exam	Needs manual entry	35.0%	
Mid-term Exam Will examine students' understanding of all course	Exam	Needs manual entry	20.0%	
permitted to take a make-up exam if its is at least ofthe course tot...	Exam	Needs manual entry	10.0%	
Quiz Used to progressively assess the students' understanding	Quiz	Needs manual entry	5.0%	

Edit Selected

Discard Incorrect Item

Confirm Selected

Confirm All Valid

SmartStudy Phase 5

4 unconfirmed item(s) are ready for review.

Figure 4. Syllabus extraction output showing detected assessment items ready for student review.

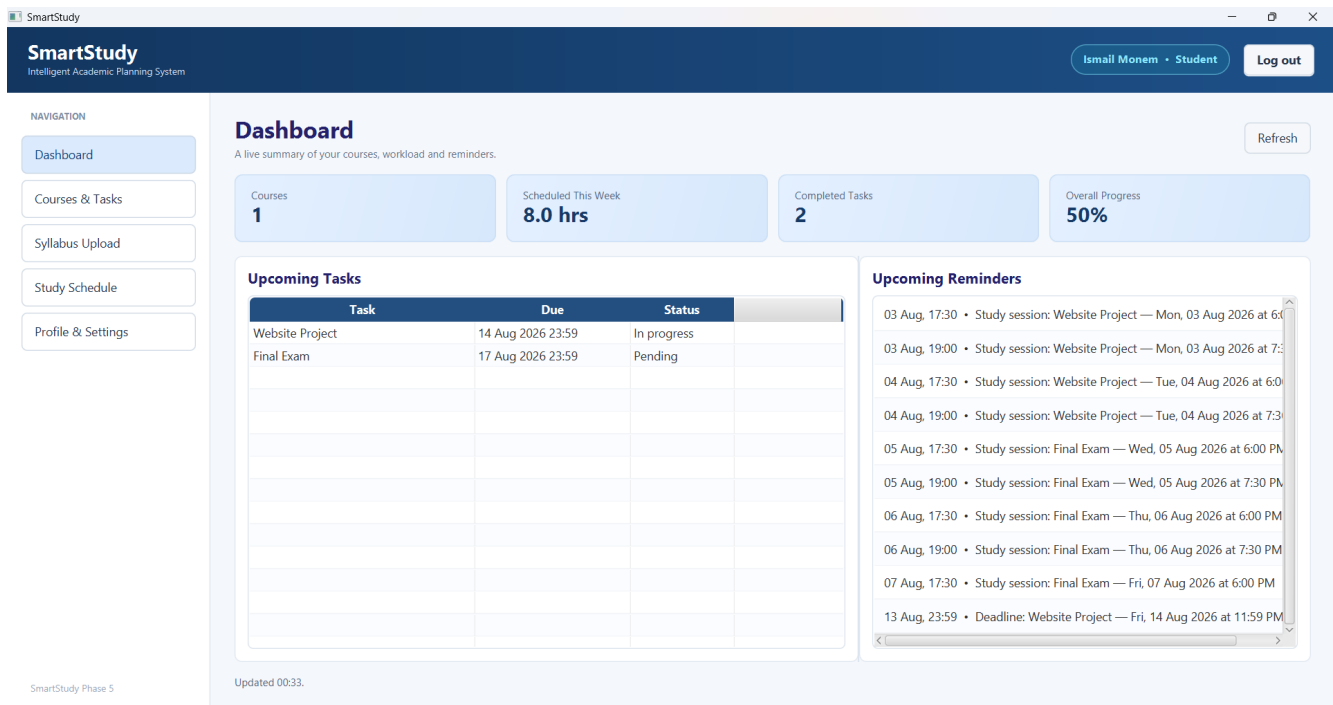


Figure 7. Ismail Monem dashboard demonstrating separate student data and 50% progress.

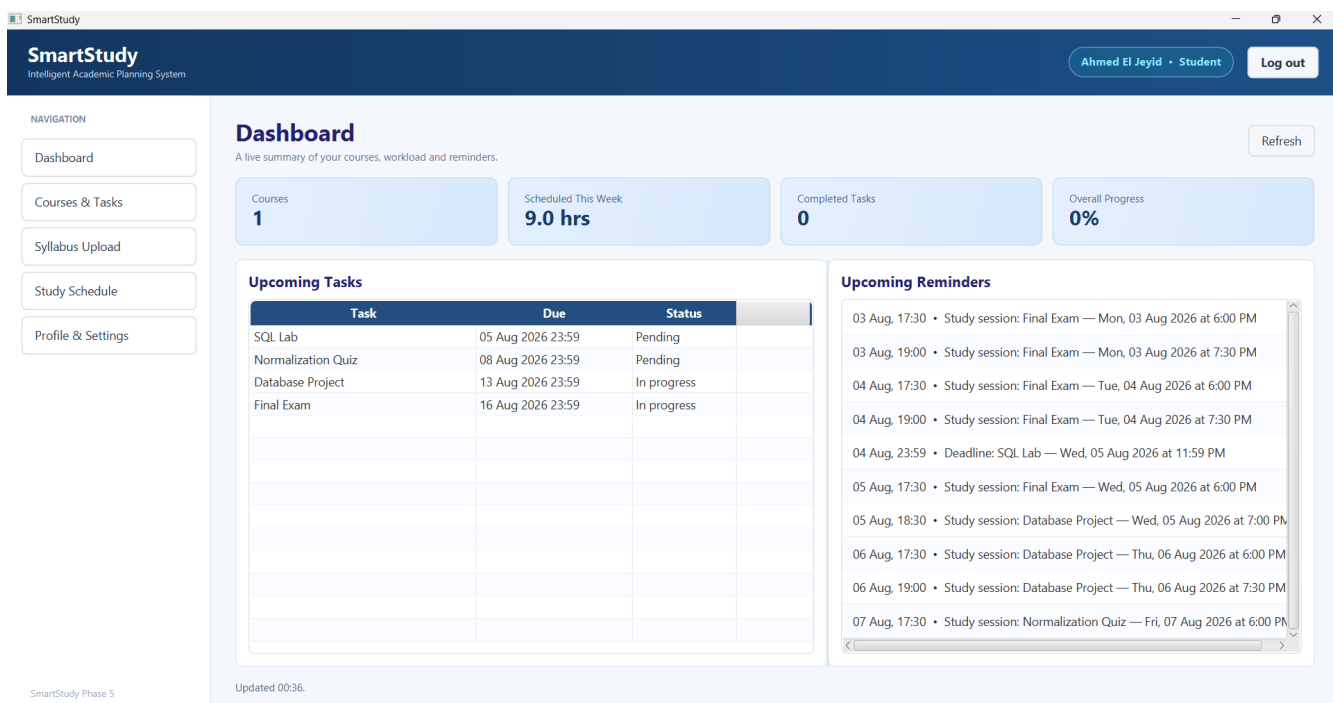


Figure 8. Ahmed El Jeyid dashboard demonstrating separate tasks, reminders, and generated workload.

SmartStudy

Intelligent Academic Planning System

System Administrator • Administrator

Log out

NAVIGATION

Administrator Console

Administrator Console

Refresh

Administrator Console

3 student account(s) loaded.

Administrator Console

3 student account(s) loaded.

Administrator Console

3 student account(s) loaded.

```
C:\Windows\System32\cmd.e  X  +  v  -  □  X

[INFO] --- surefire:3.5.0:test (default-test) @ smartstudy ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.
JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running com.smartstudy.service.ScheduleGeneratorTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.061
s -- in com.smartstudy.service.ScheduleGeneratorTest
[INFO] Running com.smartstudy.service.SyllabusParserTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.040
s -- in com.smartstudy.service.SyllabusParserTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO]
[INFO] Total time: 4.649 s
[INFO] Finished at: 2026-08-02T20:50:50+04:00
[INFO] -----
[INFO]
C:\Users\moam\Desktop\SmartStudy_Phase5_Final_Improved\SmartStudy_Phase5_Fi
nal_Improved>
```

Figure 10. Maven test output: seven tests executed with zero failures and BUILD SUCCESS.

8.8 Known Limitations and Final Validation

Limitation	Prototype handling
Institutional LMS authentication	Not included. Manual PDF/DOCX upload represents the approved prototype adapter boundary.
Email delivery	Not configured. In-app notification records and delivery status are implemented.
Unstandardized syllabi	Rule-based extraction targets representative documents; student review is mandatory.
Cloud/shared deployment	The approved prototype uses a desktop JavaFX application and local MySQL installation.
Cross-platform verification	The source is portable; the final tested configuration used Windows, Java 26, Maven 3.9.16, and MySQL 8.0.
Private repository access	The GitHub repository is private, so the instructor must be invited or visibility must be changed before grading.

Final Team Validation Checklist

- ☒ Repository URL inserted and verified.
- ☒ Private smartstudy.properties file configured and excluded from GitHub.
- ☒ Database schema and seed scripts executed in MySQL.
- ☒ mvn clean test and mvn javafx:run completed successfully.
- ☒ End-to-end functional verification completed on the team workstation.
- ☒ Real application and test screenshots included in Section 8.7.
- ☒ Default administrator account and account-management controls verified.

8.9 Team Contributions

Team Member	Implementation responsibility	Evidence to retain
Mohamed Ahmed	System integration, MySQL configuration, authentication verification, final demonstration data, schedule-fix validation, repository setup, and final testing.	Main integration commits, database/test screenshots, and repository administration.
Ahmed El Jeyid	[EDIT IF NEEDED] JavaFX interface review, navigation and usability verification, database-course demonstration data, and UI/testing documentation.	Ahmed's own GitHub commit/pull request and interface screenshots.
Ismail Monem	[EDIT IF NEEDED] Syllabus extraction review, web-design demonstration data, schedule validation, and testing documentation.	Ismail's own GitHub commit/pull request and parser/schedule evidence.

The contribution statement must match the team's actual work and Git history. The highlighted descriptions below remain editable so the team can make final corrections after Ahmed and Ismail complete their own GitHub commits.

References

Apache Maven Project. (n.d.). Maven documentation.

OpenJFX. (n.d.). Getting started with JavaFX.

Oracle. (n.d.). MySQL 8.0 Reference Manual.

Apache Software Foundation. (n.d.). Apache PDFBox and Apache POI documentation.

Provost, D., & Mazières, D. (1999). A future-adaptable password scheme.